Introduction to Programming in Python and MATLAB

Incoming M.S. Students

Structural Engineering, Mechanics and Materials

Churong Chen

University of California, Berkeley

August 2026

Contents

Roughly 60 minutes. Everything you need is free: Python via Anaconda, MATLAB via your Berkeley licence.

Contents

What is programming?

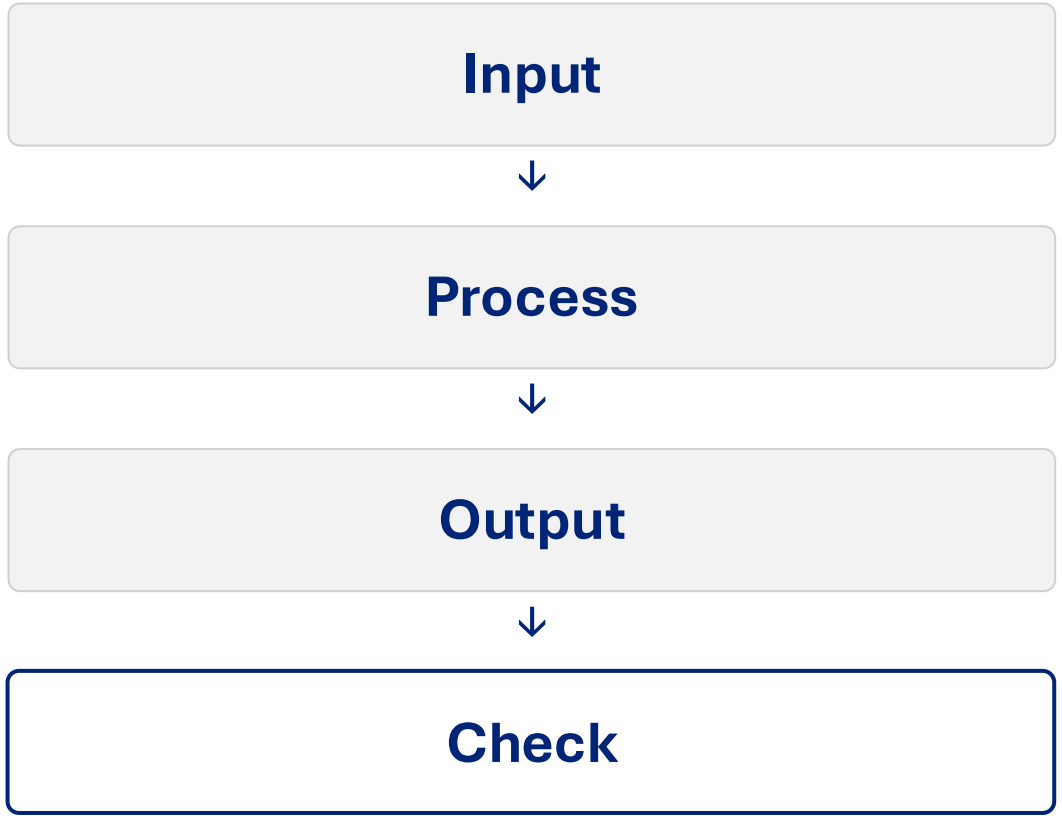
Writing a procedure down precisely enough that a machine can repeat it

- A program = data + operations on that data.
- You already do this by hand: read, compute, check, tabulate.
- The computer repeats it 10^6 times without slipping a decimal.

You gain: repeatability · scale · traceability

Why can't AI check it for you?

The loop you will live in



Why learn it now?

The work you are about to do does not fit in a spreadsheet

Parametric studies

40 sections × 8 load cases = 320 runs

Post-processing FE output

OpenSees gives you raw recorder files

Ground motion suites

11-44 records to scale and correct

Reproducibility

Somebody must be able to re-run it

Why Python? Why MATLAB?

You will end up using both

Python

- Free, and yours after graduation
- numpy, scipy, pandas, matplotlib
- OpenSeesPy, ABAQUS, ML
- You must manage environments

MATLAB

- Free now on the Berkeley licence
- Matrices are built in
- Best documentation and debugger
- Licence fees once you leave

~90% the same ideas with different punctuation. Learn one properly and the other takes a weekend.

Python vs MATLAB

Photograph this one

	MATLAB	Python
Comment	%	#
Hide output	x = 1;	x = 1
Show a value	disp(x)	print(x)
Code block	closed by end	colon + indent
Index base	1	0
Index syntax	A(i,j) , A(end)	A[i,j] , A[-1]
Power	a ^ b	a ** b
Not equal	~=	!=
Numeric array	built in	needs numpy

Contents

Installing Python

Well-developed guide to install python can be found here.

Part 1: Installing Anaconda

Since Python alone doesn't come with all the functionality required in engineering and data science, we will need to install additional packages. However, some projects require different packages and different versions of these packages. As a consequence, it's usually recommended to manage Python packages using Anaconda. Anaconda allows you to create a dedicated space for a Python installation, where you can add the packages required for your project and keep them separate from environments you use for other projects. This space is called an "environment". Anaconda will create a base environment that comes with the packages you need, but it's useful to go through the process of creating a new environment as well. Let's get started!

To install Anaconda:

1. Visit: https://www.anaconda.com/download/success?reg=skipped, select your operating system (OS), and download the graphical installer for the Anaconda Distribution.
2. Proceed with the installation. If you need specific instructions on how to install Anaconda, please refer to this guide: https://www.anaconda.com/docs/getting-started/anaconda/install/windows-gui-install (if you have a Mac, select the macOS graphical installer option in the menu on the left).

Part 2: Installing VS Code

Once we have Anaconda, we can proceed to install an editor. In reality, you can create your Python scripts using a tool as simple as Notepad in Windows. However, there are several advantages to installing a dedicated editor for coding, as it allows you to write code more efficiently (autocompletions and function highlighting, etc.), and debug and run your code. One of the most popular options is VS Code. If you are familiar with Python, you may have used Jupyter Notebooks to code; VS Code also supports creating Jupyter Notebooks (covered later in this document), and it has some advantages over directly using Jupyter Notebooks (including code completions and better highlighting). Other available editors are PyCharm and Spyder. You are free to use any tool you are comfortable with; this is just a recommended setup!

To install VS Code:

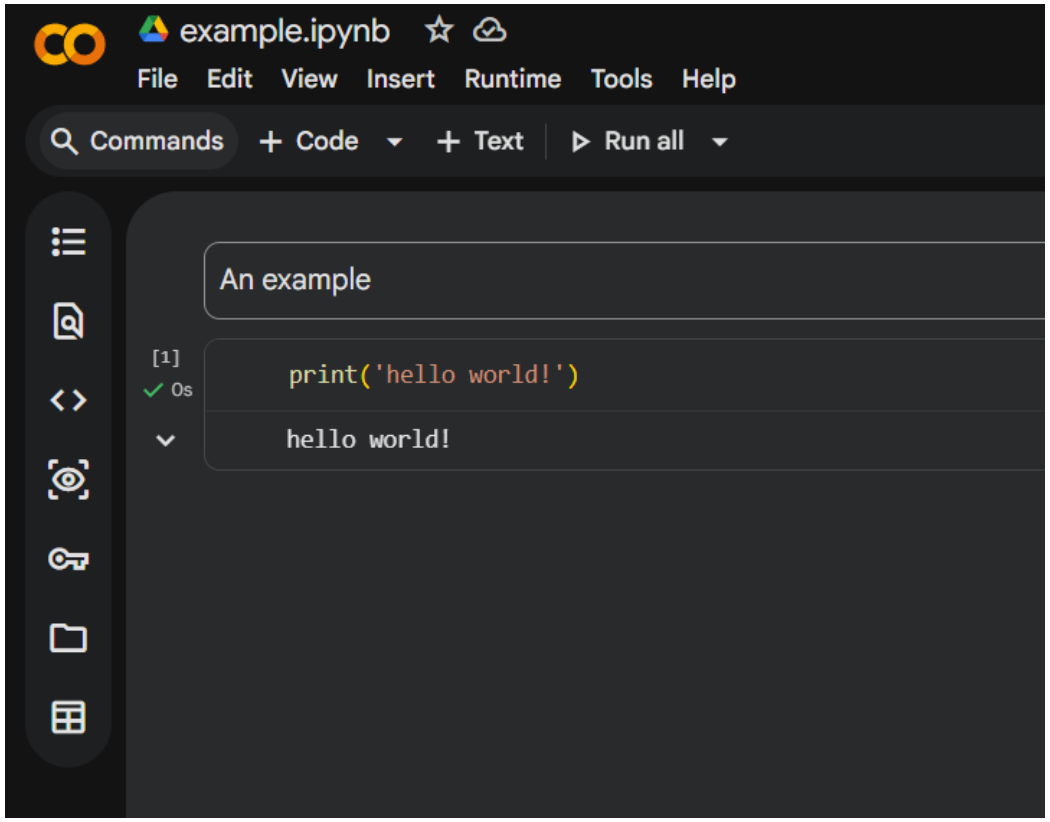
1. Visit: https://code.visualstudio.com/download?_exp_download=fb315fc982, and click on the download button for your operating system.
2. Proceed with the installation. Specific instructions, depending on your operating system, can be found at this link: https://code.visualstudio.com/docs/setup/windows. Same as before, if your OS is not Windows, switch in the menu on the left.

*An easier option is to use **Google Colab** to create Jupyter Notebooks and run your analysis in the cloud.*

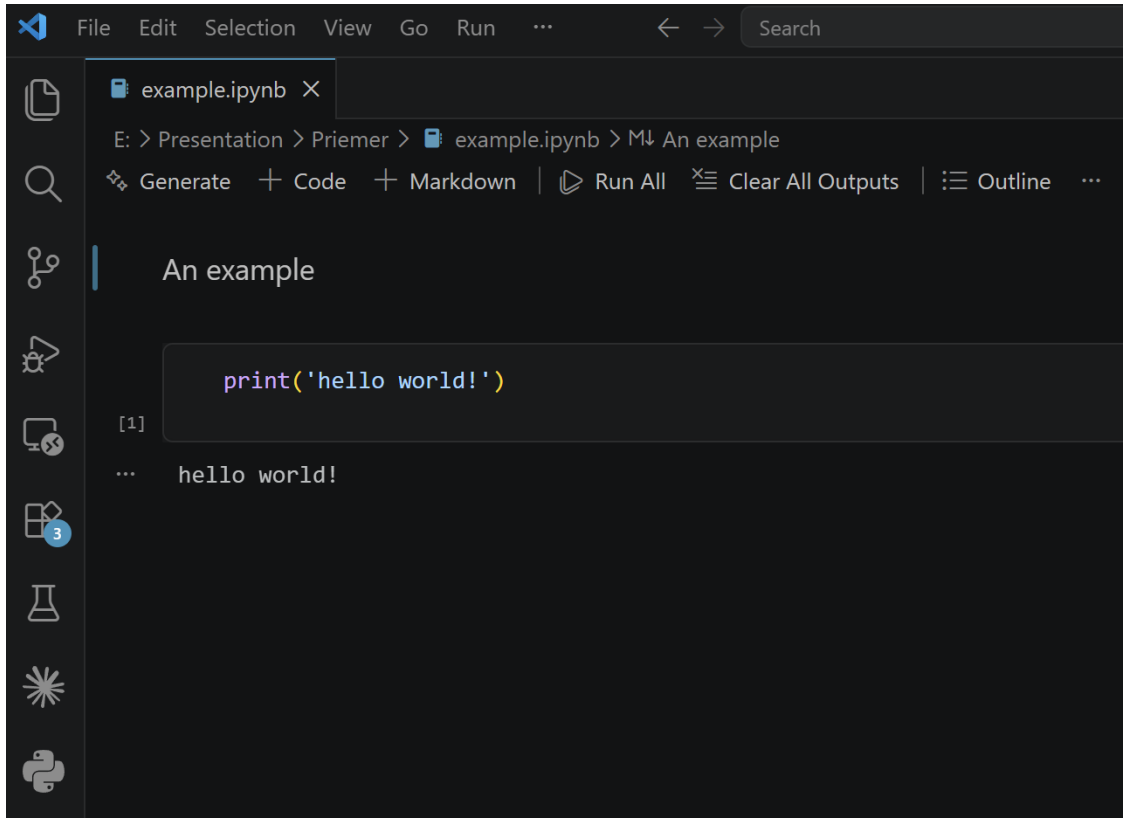
Jupyter in Google Colab & VS Code

https://colab.research.google.com/

Google Colab



VS Code



.ipynb vs .py

Both are Python. The difference is how they run.

.ipynb notebook

- Cells, with output stored inline
- Exploration and showing results
- Risk: hidden state

.py script / module

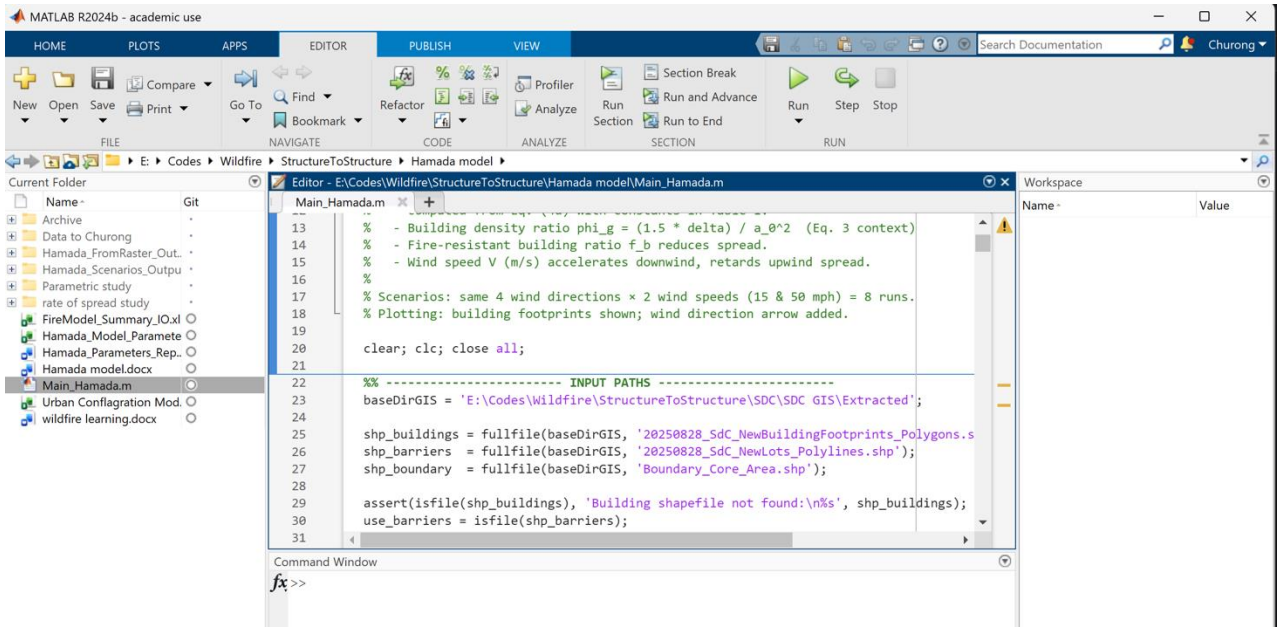
- Plain text, runs top to bottom
- The only thing you can import
- No inline output

- Explore in .ipynb , move working code into .py , import it back.
- **Restart Kernel and Run All before you trust any notebook result.**

Setting up MATLAB

Free through Berkeley's campus licence — activate it early

- MathWorks account with your @berkeley.edu email
- Activate the licence at software.berkeley.edu
- Install it, or use MATLAB Online — nothing to install



Panes: Current Folder · Editor · Workspace · Command Window

MATLAB only sees files in the Current Folder — set it first.

Contents

Variables, types and printing

Python

```
fy   = 50.0          # float
n    = 12            # int
name = "W14x90"      # str
ok   = fy > 36       # bool

print(f"{fy:.1f} ksi")
```

MATLAB

```
fy   = 50.0;         % double
n    = 12;           % double
name = "W14x90";     % string
ok   = fy > 36;      % logical

fprintf('%.1f ksi', fy)
```

- **f-strings are the formatting trick you will use daily: f"{value:.3f}"**
- Name variables after the physics — fy , not x1 .

Lists, tuples and nested lists

Flexible containers — but not for maths

```
loads = [12.5, 18.0, 9.75]      # list, square brackets
loads.append(22.0)              # grows in place
loads[0]                        # 12.5

node = (10.0, 0.0, 13.0)        # tuple, immutable
x, y, z = node                  # unpacking

frame = [[1, 2], [3, 4]]        # nested = list of lists
frame[1][0]                     # 3
```

Lists do not do maths.

```
[1, 2] + [3, 4]  →  [1, 2, 3, 4]      concatenation, not addition
```

Indexing and slicing

The #1 source of silent off-by-one errors

	Python	MATLAB
First element	a[0]	a(1)
Last element	a[-1]	a(end)
First three	a[:3]	a(1:3)
Everything	a[:]	a(:)
Reversed	a[::-1]	flip(a)
2-D element	A[i, j]	A(i, j)
Whole row	A[i, :]	A(i, :)

a[:3] gives three items. Python slices stop BEFORE the end index; MATLAB includes it.

Why NumPy

MATLAB has arrays built in. Python imports them.

Plain lists

```
a = [1, 2, 3]
b = [4, 5, 6]

a + b    # [1,2,3,4,5,6]
a * 2    # [1,2,3,1,2,3]
a / 2    # TypeError
```

numpy arrays

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b    # [5, 7, 9]
a * 2    # [2, 4, 6]
a @ b    # 32
```

- Creating them: np.zeros(n) · np.arange(0,10,0.5) · np.linspace(0,10,21)
- **Any numerical vector or matrix → numpy array, never a list.**

NumPy ↔ MATLAB

Same operations, different spelling

Task	MATLAB	numpy
Zeros	zeros(3,4)	np.zeros((3,4))
n points	linspace(0,10,21)	np.linspace(0,10,21)
Shape	size(A)	A.shape
Transpose	A'	A.T
Element-wise ×	**A .* B**	**A * B**
Matrix ×	**A * B**	**A @ B**
Solve A x = b	A \ b	np.linalg.solve(A,b)
Max and index	[m,i] = max(v)	v.max() , v.argmax()

The two bold rows are swapped between the languages — and produce a wrong number, not an error.

if / elif / else

Same logic, different block syntax

Python

```
if dcr > 1.0:
    status = "FAIL"
elif dcr > 0.90:
    status = "check"
else:
    status = "OK"
```

MATLAB

```
if dcr > 1.0
    status = "FAIL";
elseif dcr > 0.90
    status = "check";
else
    status = "OK";
end
```

- **In Python the indentation is the syntax — four spaces, consistently.**
- `==` compares, `=` assigns. Never mix them up.

Loops

The point of a computer

Python

```
drifts = [0.004, 0.011, 0.019]

for i, d in enumerate(drifts):
    if d > 0.015:
        print(i + 1, d)
```

MATLAB

```
drifts = [0.004 0.011 0.019];

for i = 1:length(drifts)
    if drifts(i) > 0.015
        disp(i)
    end
end
```

Putting it together

A complete story-drift check

```
import numpy as np

h     = np.array([13.0, 12.0, 12.0, 12.0])
delta = np.array([0.60, 1.05, 1.55, 1.90])
limit = 0.020

drift = np.diff(delta, prepend=0.0) / (h *
12)
for i, dr in enumerate(drift):
    flag = "FAIL" if dr > limit else "OK"
    print(i + 1, round(dr, 4), flag)
```

Output

```
1 0.0038 OK
2 0.0031 OK
3 0.0035 OK
4 0.0024 OK
```

Arrays, arithmetic, a loop, a conditional — all of section 3 in ten lines.

Contents

Importing packages

You will rarely write numerical code from scratch

```
import numpy as np                      # always np
import pandas as pd                     # always pd
import matplotlib.pyplot as plt         # always plt

from scipy.optimize import fsolve       # one name only
from pathlib import Path
```

- Stick to the community aliases — every example online uses them.
- **ModuleNotFoundError usually means right package, wrong environment.**

Importing your own .py files

When the notebook gets long, move functions out

Layout

```
drift_study/
  data/
  results/
  sections.py
  run.ipynb
```

sections.py

```
def moment_capacity(fy, Zx, phi=0.9):
    Mn = fy * Zx
    return Mn, phi * Mn

# then, in run.ipynb:
from sections import moment_capacity
```

- Any .py file in the same folder imports by its filename.
- **Keep raw data in data/ and never write to it.**

Contents

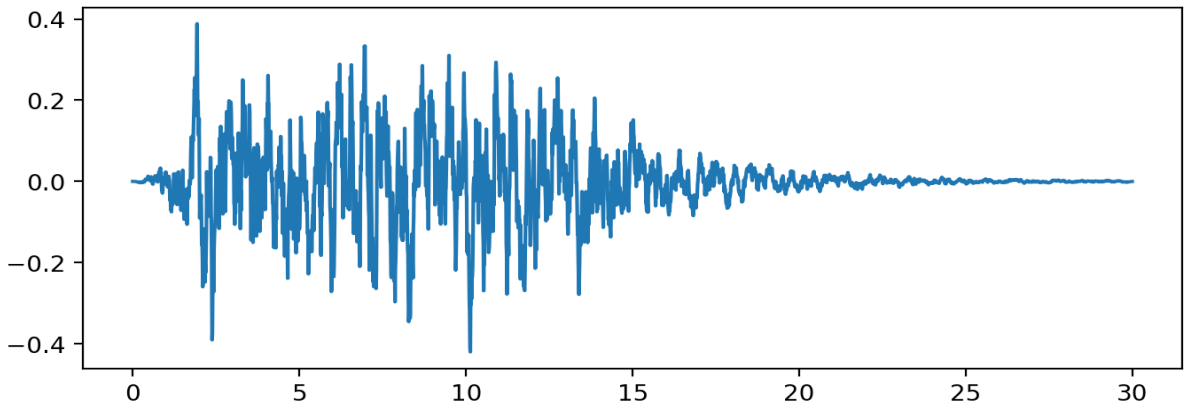
Your first plot

Three lines to see your data

```
import matplotlib.pyplot as plt

t = np.linspace(0, 30, 3000)
a = np.loadtxt("record.txt")

fig, ax = plt.subplots()
ax.plot(t, a)
plt.show()
```



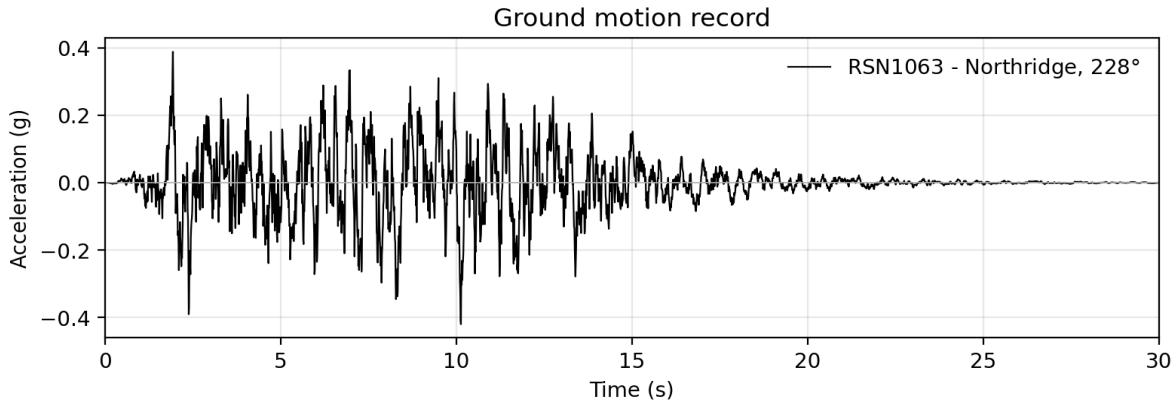
Unlabelled, unreadable, unpublishable.

- fig is the canvas, ax is one set of axes.

Formatting

What turns a plot into a figure

```
ax.plot(t, a, color="k", lw=0.8)
ax.set_xlabel("Time (s)")
ax.set_ylabel("Accel (g)")
ax.set_xlim(0, 30)
ax.legend(frameon=False)
ax.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("gm.png", dpi=300)
```



- Axis labels with units — every time
- A legend if there is more than one series
- **Never screenshot. Always savefig() .**

Multiple panels

Stacked axes sharing a time base

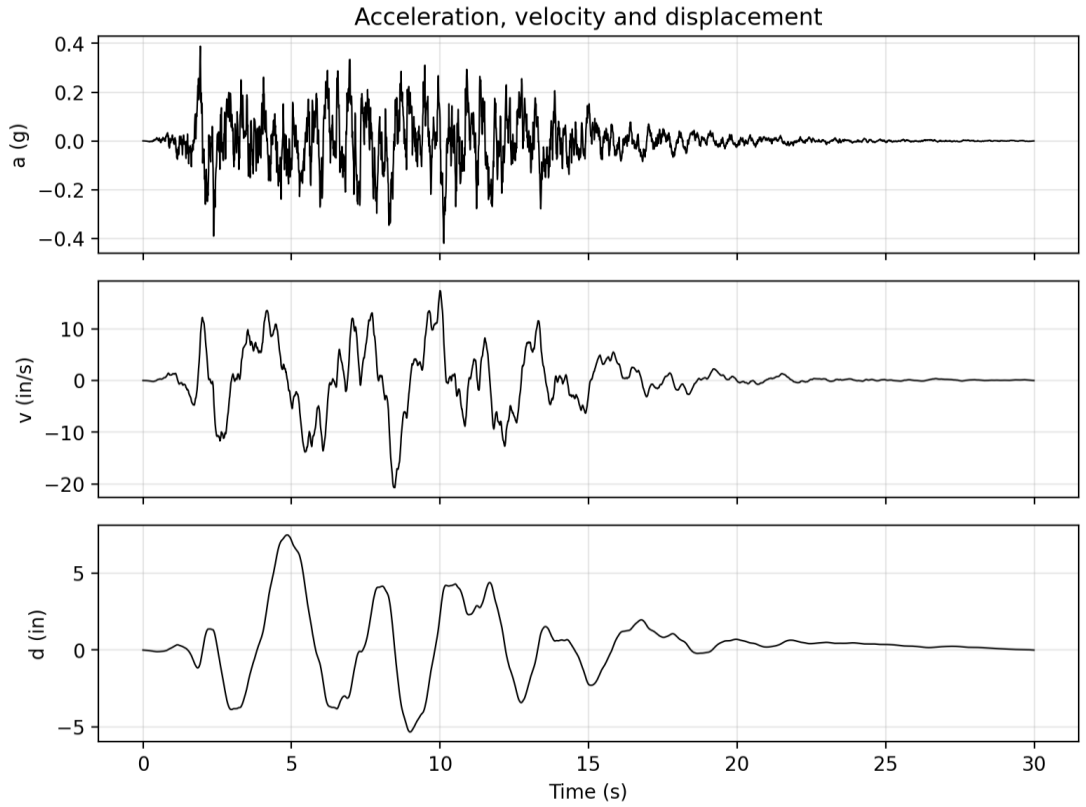
```
fig, axes = plt.subplots(
        3, 1, sharex=True)

for ax, y in zip(axes, [a, v, d]):
    ax.plot(t, y, color="k")
    ax.grid(alpha=0.3)

axes[-1].set_xlabel("Time (s)")
```

MATLAB: subplot(3,1,1); plot(t,a)

Also useful: semilogx · scatter · fill_between



Contents

File input and output

Nobody types a ground motion record by hand

- One PEER record: a few header lines, then ~8,000 values.
- A code suite: 11-44 records × 2-3 components.
- One OpenSees run writes a recorder file per quantity.

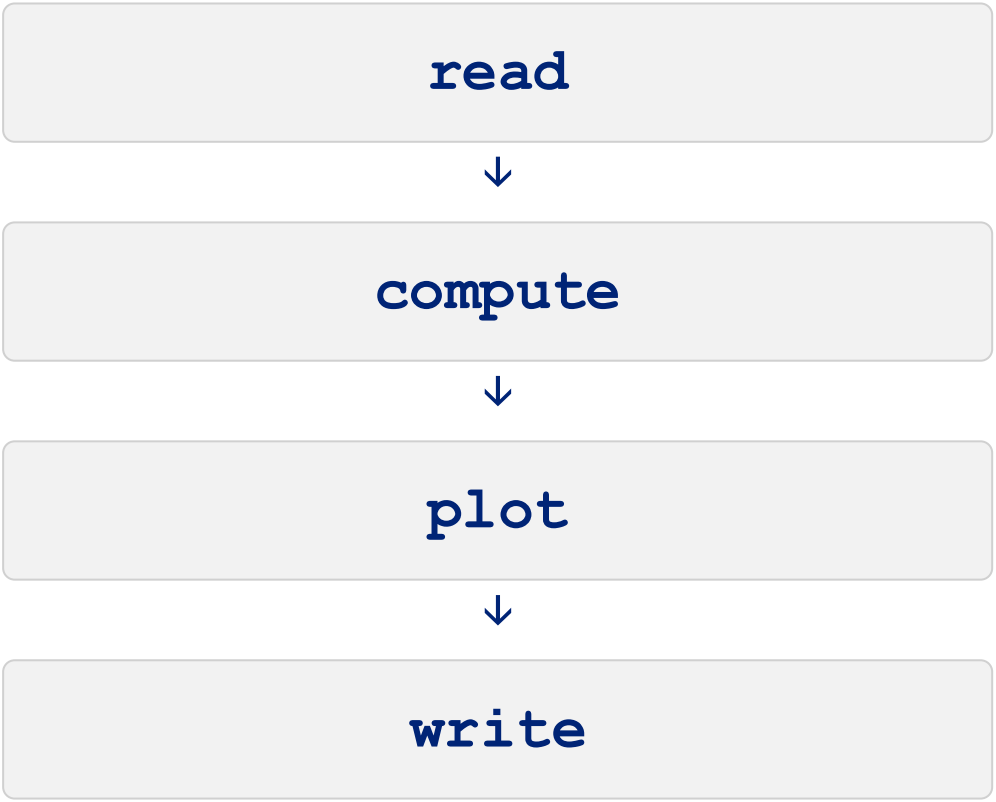
pandas.read_csv

columns with names → a DataFrame

numpy.loadtxt

a pure grid of numbers → an array

Every script you write



Reading data in

numpy — a grid of numbers

```
a = np.loadtxt("rec.txt",
               skiprows=4)
a = a.ravel()

a.shape
a.max()
```

pandas — named columns

```
df = pd.read_csv("drifts.csv")

df.head()
df["drift"].max()
df[df["drift"] > 0.02]
df.describe()
```

- **Always df.head() a file the first time you open it.**
- Check the row count, the column names and the units before you trust it.

Writing results out

If it is not written to a file, it did not happen

```
out = pd.DataFrame({"story": [1, 2, 3, 4],
                    "drift": drift})
out.to_csv("results/check.csv")

np.savetxt("results/drift.txt", drift)

with open("results/summary.txt", "w") as f:
    f.write(f"max {drift.max():.4f}")
```

MATLAB

```
readmatrix(f)
readtable(f)

writematrix(d, f)
writetable(T, f)
```

- **Read from data/ , write to results/ . Never overwrite raw data.**
- Use relative paths, or your code runs on exactly one computer.

The whole thing

Read → compute → plot → write, over a record suite

```
rows = []
for f in sorted(Path("data").glob("*.AT2")):
    a = np.loadtxt(f, skiprows=4).ravel()
    t = np.arange(len(a)) * 0.005

    rows.append({"rec": f.stem,
                 "PGA": np.abs(a).max()})

    fig, ax = plt.subplots()
    ax.plot(t, a, "k", lw=0.7)
    fig.savefig(f"figs/{f.stem}.png")
    plt.close(fig)

pd.DataFrame(rows).to_csv("pga.csv")
```

- a loop over files
- numpy maths
- a figure, saved
- a csv written out

Drop one more record into data/ and nothing else changes.

Where to go next

You will not remember syntax. Know where to look.

Learn

- docs.python.org tutorial
- MATLAB Onramp — 2 h, free
- matplotlib.org gallery

Habits

- One folder per project
- Functions in .py
- git init on day one
- Read the LAST error line first

Questions?